What building these graphs found

Writing 22 graphs against every craft pack turned up gaps in PlumCraft and in the packs. Each entry says what happened, how the graphs work around it today, and where the fix belongs. They are the maintainer's repositories, so each should be fixed there, not papered over here.

Fixed in PlumCraft

Headless runs couldn't run these graphs

Symptom: `plumcraft-editor run FILE` stopped with "run.save is not available: this engine doesn't save files" on every graph here. Headless `run` only wrote through `output.save`, which only engines with save formats offer. The craft, array and audio engines don't, so a graph ending in Write Files couldn't run without a window.

Effect: the first version of this project carried its own Rust runner (`plumgraph`) to start the engines, put them behind a coordinator and run the Write Files nodes. The maintainer's verdict: "If this needs a custom runner, it suggests that there is something missing from plumcraft base."

Fix (PlumCraft, branch `headless-write-files`, not yet released):

- `run FILE` runs the workflow's writing nodes (`output_node`: Write Files, Save WAV) when it has no Result node. `--out DIR` points their folders there.
- `run`, `check` and `tidy` take their engines from the file's `engines` list when no `--engine` is given.
- New `check FILE` reports unknown node types and inputs, wires the type rules refuse, wire-only inputs left unwired, and nodes no output depends on.
- New `tidy FILE` runs Edit › Tidy and saves.

The runner was then deleted from this project. `scripts/graphs` is a thin loop over those commands. The two checks that were new in `check` each caught real mistakes in these graphs: a background-delete node that was never wired in, and four query nodes whose results went nowhere.

GridCraft's Export PDF exported only the active sheet

Symptom: a four-sheet workbook came out as one page, the last sheet's. The pack's `gc.export_pdf` used `file.saveAs`, which saves the active sheet.

Fix (PlumCraft, `crates/craft/packs/gridcraft.toml`): it now runs `file.exportPdf`, which honours each sheet's Page Setup, with a `sheets` choice: `active` (the default, as before) or `all`. The other GridCraft graphs give the same pages as before.

Open: PlumCraft core

A node's `id` output can't feed another node's `ids` input

DesignCraft's `dsc.shape` and `dsc.text_frame`, and FigCraft's shape nodes, output the new object's `id` as `INT`. Every `select`-taking node has an `ids` socket of type `JSON`, and there's no `INT → JSON` conversion, so the obvious wiring ("rotate the frame I just made") is refused: `type mismatch: tab0.id is INT, turn0.ids is JSON`.

Workaround here: order the chain so that `select: all` only matches what's intended (`lost-sock-poster`, `design-to-code`), or select by a literal id in a freshly built document (`ids:10`), which breaks if the craft numbers objects differently.

Fix: allow `INT → JSON` (a one-element list) in the compat table, or type the `id` outputs as `JSON`.

TXT and MD aren't TEXT

`pdf.export_text` outputs `TXT` and `wc.export_md` outputs `MD`. Neither converts to the core `TEXT` type, so a PDF's text can't go to the text engine (`x.text is TXT, l.text is TEXT`).

Fix: have the craft adapter type plain-text exports as `TEXT`, or add `TXT → TEXT` and `MD → TEXT` to the compat table.

Open: the craft packs (`crates/craft/packs`)

Seven Open nodes offer an `image` input that fails at run time

Every craft's `*.open` has an `image` socket, but only PhotoCraft, VectorCraft and FigCraft accept a picture. The others fail when the graph runs:

Open node	Error
`dk.open`	not a presentation DeckCraft can read
`dsc.open`	not a DesignCraft or IDML file
`wc.open`	unsupported format `.png`
`ec.open`	not a text project file
`cad.open`	unsupported file type `png`
`sol.open`	design file is not UTF-8 JSON
`pdf.open`	the file is not a readable PDF

Fix: drop the socket from those packs, or make it place the image (DeckCraft and DesignCraft both have place-picture commands; PdfCraft can make a page from an image). Placing would open up the obvious crossings: a SolidCraft render into a DesignCraft spec sheet, an array plot into a DeckCraft slide.

`select` filters that a craft can't do are offered anyway

The `select` tooltip lists `name:`, `kind:` and `color:` for every craft. DesignCraft, FigCraft and CadCraft have no `[selection] find`, so `select: name:Onboarding card` fails at run time with "this craft can't find objects". `check` can't tell either.

Fix: offer only the forms the pack supports, or add `find` to those packs.

WordCraft's mail merge can't be run from a graph

`mailings.*` isn't in `wc.run`'s allow list, so a table of recipients can't be merged into a letter. A template can be filled with `edit.replaceAll` instead, one node a placeholder.

Fix: allow `mailings.start`, `mailings.recipients` and `mailings.finish` (it writes a file only when given a path).

Open: the craft apps

App	Finding	Workaround here
GridCraft	`chart.set` without `title` resets the title to "Chart Title"	pass the title again
GridCraft	hidden columns still print as blank pages in Export PDF	`pageLayout.printArea`
GridCraft	`home.autofitColumnWidth` with no `cols` fitted column A only	name the columns
DesignCraft	`dsc.equation` ignores `x`/`y` and goes inline into the most recent text frame	give it its own frame first
EffectCraft	the composition background isn't in `ec.export_png`; decoded frames come out black	a solid layer at the bottom

App	Finding	Workaround here
CadCraft	Export SVG includes a white background rectangle; imported into VectorCraft it becomes geometry (a laser would cut it)	`vc.run edit.clear` on `color:#ffffff`
CadCraft	Export PDF and SVG are fitted to the page, not 1:1	documented; the DWG is true scale
FigCraft	`fills` in `props` takes `["#RRGGBB"]`, not the `{type, color}` paints its error message suggests	plain colours
SolidCraft	each run of the die takes about 10 s (21 sphere cuts)	none needed

Things that worked better than expected

- **Crossing engines:** values crossed borders unchanged in every crossing (images, SVG, WAV, JSON rows, arrays). The coordinator needed no help, and building-block nodes ran in whichever engine they were pinned to.
- **Error messages:** the craft errors usually said exactly what to do ("no swatch named `Paper` (did you mean `[Paper]`?)"), which made most fixes one edit.
- `gc.read` → `cad.pline`: `[[x, y], …]` rows from a spreadsheet are already a CAD polyline. That one wire is what makes the garden planner work.